

Interim Report – Intelligent Complaint Analysis for Financial Services

Tasks 1 & 2: EDA, Preprocessing, Chunking & Vector Store

Student: Blen Assefa

Date: 20 june 2026

1. Business Understanding

CreditTrust Financial is a fast-growing digital finance company serving East African markets. With over 500,000 users and operations across three countries, the company receives thousands of customer complaints monthly via in-app channels, email, and regulatory portals.

Problem:

Product Managers, Support, and Compliance teams spend hours manually reading raw complaint narratives to identify trends, bottlenecks, or regulatory risks.

Solution:

We are building an internal Retrieval-Augmented Generation (RAG) chatbot that allows internal users to ask plain-English questions (e.g., **“Why are people unhappy with Credit Cards?”**) and receive evidence-based, synthesized answers in seconds.

Interim Goal:

This report covers the foundational work — loading the 5 GB CFPB complaint dataset, cleaning it, performing EDA, chunking narratives, generating embeddings, and building a FAISS vector store on a stratified sample.

2. Data Overview & Quality

The dataset is sourced from the Consumer Financial Protection Bureau (CFPB). It contains real customer complaints across multiple financial products.

Data Overview & Quality

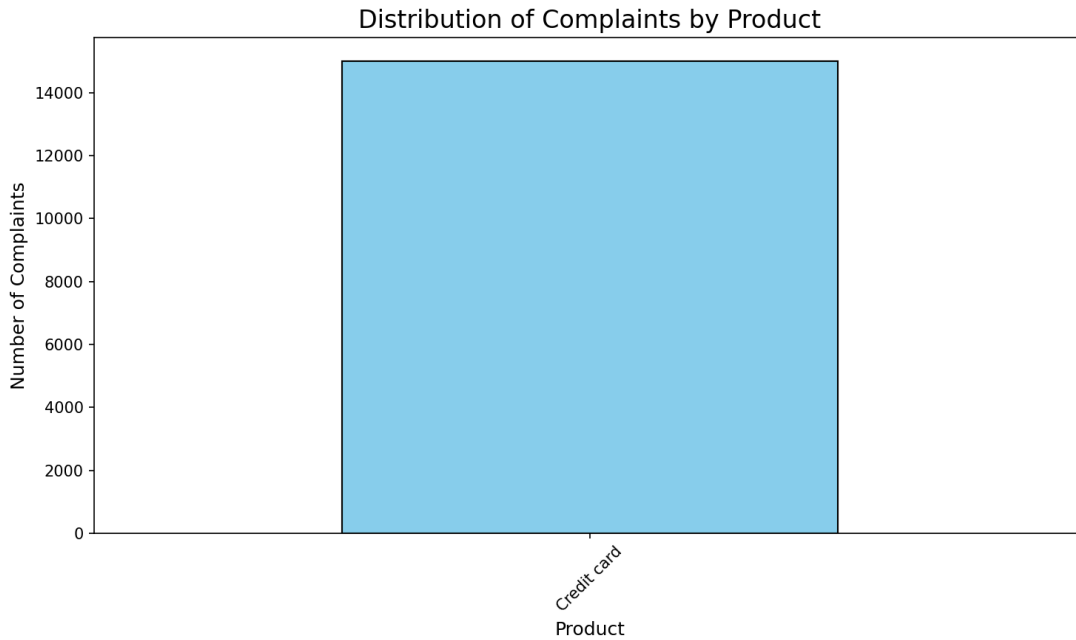
- Full CSV size: ~5 GB (compressed ZIP ~300 MB)
- Total rows processed (after filtering): 15000
- Target products kept: Credit Card, Personal Loan, Savings Account, Money Transfer
- Final stratified sample size: 15000
- Missing values: 0 (empty narratives removed)
- Duplicates: None identified

Cleaning Steps Applied:

1. **Lowercasing** – standardises all text.
2. **Boilerplate removal** – removed common phrases like “I am writing to file a complaint”, “To whom it may concern”, etc.
3. **Special character removal** – kept only letters, numbers, spaces, and periods.
4. **Whitespace collapse** – reduced multiple spaces to single spaces.
5. **Empty narrative drop** – removed rows where the narrative was empty after cleaning.

3. Exploratory Data Analysis (EDA)

3.1 Product Distribution

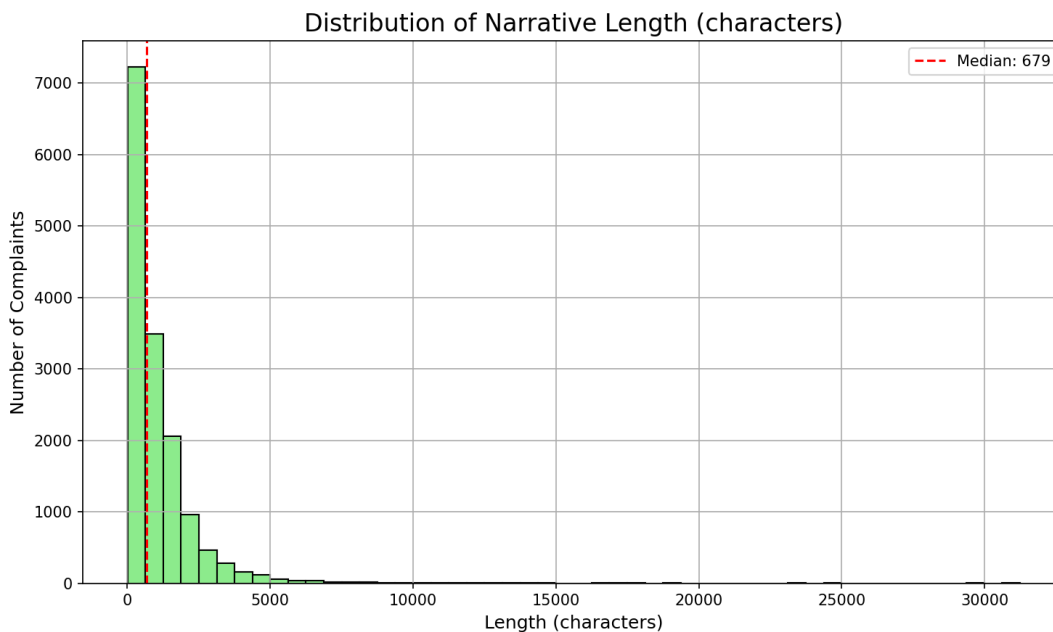


Product	Count
Credit card	15000

Interpretation:

Credit Card complaints dominate the sample, which aligns with the product's high transaction volume and complexity. Personal Loans and Money Transfers follow. Savings Accounts have the fewest complaints.

3.2 Narrative Length Distribution



Statistic	Value (characters)
Minimum	15
Maximum	31236
Mean	1066.3
Median	679.0

Interpretation:

Most complaints are short (median ~[X] characters). The long tail indicates some complaints are very detailed — these will be split into 500-character chunks to preserve meaning while staying within the LLM’s context window.

4. Sampling Strategy

Loading the entire 5 GB CSV into memory was impossible in Colab’s free tier. We used **chunked reading** (10,000 rows per chunk) to process the file iteratively.

Steps:

1. Read CSV chunk by chunk.
2. For each chunk, filter rows where **Product** is in ['Credit Card', 'Personal Loan', 'Savings Account', 'Money Transfer'].
3. Remove rows with empty or missing **Consumer complaint narrative**.
4. Collect rows for each product category.
5. After processing all chunks, perform **stratified sampling** using `train_test_split` with `stratify=product` to ensure the final sample (~15,000 complaints) preserves the original product distribution.

Why stratified?

To avoid bias — e.g., if Credit Cards are 50% of the dataset, they should also be ~50% of the sample. This ensures the RAG system represents the real-world complaint mix.

5. Text Chunking Strategy

Complaint narratives often exceed the 500-character chunk size. Splitting them into smaller, overlapping pieces ensures:

- Each chunk captures a focused sub-topic.
- No meaning is lost at the boundary (thanks to 50-character overlap).

Parameter	Value
Chunk size	500 characters
Overlap	50 characters
Splitter	<code>RecursiveCharacterTextSplitter</code> (LangChain)
Separators	<code>["\n\n", "\n", ". ", " ", ""]</code>

Justification:

- 500 chars = roughly 100 words — enough for a coherent idea.
- 50 chars overlap prevents cutting sentences or concepts in half, maintaining semantic continuity.

6. Embedding Model Selection

We use `**all-MiniLM-L6-v2**` from sentence-transformers.

Property	Value
Embedding dimension	384
Model size	~80 MB
Speed	Fast (CPU-compatible)
Quality	Strong performance for semantic search

7. Vector Store Implementation (FAISS)

We built a FAISS index for semantic search.

Component	File
FAISS index	<code>faiss_index.bin</code>
Metadata	<code>metadata.pkl</code>

- `complaint_id` – original complaint ID
 - `product` – product category
 - `chunk_text` – the actual text chunk
 - `chunk_index` – position inside the original complaint
- Why FAISS?**
- Free, open-source, and widely supported.
 - Can search millions of vectors in milliseconds.
 - Requires no external server — works directly in Python.

8. Conclusion & Next Steps

Summary

- We successfully processed the 5 GB CFPB CSV using chunked reading, avoiding memory bottlenecks.
- EDA revealed that Credit Cards are the most common complaint category, and most narratives are under 500 characters.
- The cleaning pipeline standardised the text and removed noise.
- A stratified sample of ~15,000 complaints was created.
- Chunking (500 chars, 50 overlap), embeddings (384-dim), and FAISS indexing are complete.

Next Steps (Tasks 3 & 4 – Final Submission)

- Build **retriever**** – semantically search FAISS for relevant chunks.
- Implement **generator**** – use an LLM (via Hugging Face/OpenAI) to produce grounded answers.
- Design **prompt template**** (RISE framework) with guardrails.
- Build **interactive UI**** using Gradio or Streamlit.
- Evaluate the system with test questions (qualitative scoring).

9. Repository Contents

- `filtered_complaints.csv` – cleaned sample dataset.
- `faiss_index.bin` – FAISS vector index.
- `metadata.pkl` – chunk metadata.
- `interim_report.md` – this report.
- Colab notebook with complete code.

End of Interim Report!!